

Spółeczna Akademia Nauk
Instytut Technologii Informatycznych

Kierunek studiów:
INFORMATYKA

Przedmiot:
Systemy Szkieletowe

UniRoom (Booking_SAN)

System rezerwacji sal uniwersyteckich

Dmytro Kovalov

Nr albumu: 121262

Pavlo Krasnivskyi

Nr albumu: 121547

Grupa: 3

Prowadzący:
mgr inż. Krystian Gumiński

Łódź 2026

Spis treści

1	Wstęp	3
1.1	Zespół projektowy i podział odpowiedzialności	3
2	Cel projektu	5
2.1	Wymagania funkcjonalne	5
2.2	Wymagania niefunkcjonalne	5
3	Przegląd istniejących rozwiązań	7
3.1	Rozwiązanie A: Kalendarz Google z zasobami	7
3.2	Rozwiązanie B: Skrzynki zasobów Exchange / Outlook	7
3.3	Rozwiązanie C: Wyspecjalizowane platformy rezerwacji (np. Skedda)	7
4	Opis architektury	9
4.1	Podział na warstwy	9
4.2	Przepływ danych – przykład rezerwacji sali	10
5	Opis implementacji	11
5.1	Frontend	11
5.1.1	Konfiguracja i stylizacja	11
5.1.2	Stan sesji i uwierzytelnianie	11
5.1.3	Strony aplikacji	12
5.1.4	Model sal i przedziałów czasowych	12
5.2	Backend	12
5.2.1	Moduł db.py	12
5.2.2	Klient REST api_client.py	13
5.2.3	Usługa ASP.NET Web API	14
5.3	Baza danych	14
5.3.1	Struktura tabel	15
5.3.2	Integralność danych	15
5.3.3	Widoki i funkcje	15
5.4	Docker	16
5.4.1	Obraz aplikacji (Dockerfile)	16
5.4.2	Orkiestracja (docker-compose.yml)	16
5.4.3	Uruchomienie	16
5.5	Testy	17
5.5.1	Zakres i poziomy testów	17
5.5.2	Scenariusze testowe	17

5.6	Monitoring	19
6	Podsumowanie i wnioski	20
6.1	Rezultaty	20
6.2	Napotkane problemy i sposoby ich rozwiązania	20
6.3	Kierunki dalszego rozwoju	20

1 Wstęp

Niniejsza dokumentacja opisuje projekt **UniRoom** (nazwa repozytorium: `Booking_SAN`), czyli aplikację internetową umożliwiającą studentom rezerwację sal dydaktycznych na uczelni. Projekt został zrealizowany w ramach przedmiotu *Systemy Szkieletowe* i obejmuje pełny cykl wytwarzania niewielkiego systemu informatycznego: od zaprojektowania bazy danych, przez implementację warstwy logiki i interfejsu użytkownika, po konteneryzację i uruchomienie całego środowiska za pomocą jednego polecenia [1].

Aplikacja pozwala na założenie konta studenta, zalogowanie się, przeglądanie katalogu sal wraz z ich wyposażeniem, sprawdzanie dostępności terminów oraz rezerwowanie i anulowanie rezerwacji. Dane użytkowników i rezerwacji są przechowywane w relacyjnej bazie danych PostgreSQL, a całość środowiska (aplikacja i baza) jest uruchamiana w kontenerach Docker.

W dalszej części dokumentu omówiono cel projektu (rozdział 2), przegląd istniejących rozwiązań (rozdział 3), architekturę systemu (rozdział 4), szczegóły implementacji poszczególnych komponentów (rozdział 5) oraz podsumowanie wraz z wnioskami i kierunkami dalszego rozwoju (rozdział 6).

1.1 Zespół projektowy i podział odpowiedzialności

Projekt został wykonany przez dwuosobowy zespół. Podział odpowiedzialności za poszczególne części systemu przedstawia tabela 1. Obszar *Testy* był wspólną odpowiedzialnością obu autorów: każdy z nich testował komponenty, za które odpowiadał, a wspólnie zweryfikowano działanie całości (por. podrozdział 5.5).

Tabela 1: Podział odpowiedzialności w zespole projektowym

Autor	Nr albumu	Zakres odpowiedzialności
Dmytro Kovalov	121262	Frontend, Backend, Testy
Pavlo Krasnivskyi	121547	Baza danych, Docker, Testy

Szczegółowe przyporządkowanie plików repozytorium do autorów przedstawia tabela 2.

Tabela 2: Przyporządkowanie elementów repozytorium do autorów

Obszar	Elementy repozytorium	Odpowiedzialny
Frontend	<code>main.py</code> (interfejs Streamlit, style CSS, walidacja formularzy)	D. Kovalov
Backend	<code>db.py</code> , <code>api_client.py</code> , katalog API/ (ASP.NET Web API)	D. Kovalov
Baza danych	<code>Boo.sql</code> (schemat, dane słownikowe, widoki, funkcje, wyzwacze)	P. Krasnivskyi
Docker	<code>Dockerfile</code> , <code>docker-compose.yml</code> , <code>.env</code> , <code>requirements.txt</code>	P. Krasnivskyi
Testy	scenariusze testowe, weryfikacja komponentów i całości	obaj autorzy

2 Cel projektu

Celem projektu jest zaprojektowanie i wykonanie aplikacji, która eliminuje konieczność ręcznego, papierowego lub mailowego uzgadniania rezerwacji sal na uczelni. Aplikacja ma dawać studentowi natychmiastową informację o tym, które sale i w jakich terminach są wolne, oraz pozwalać na złożenie rezerwacji w kilku krokach, bez udziału administracji.

2.1 Wymagania funkcjonalne

- rejestracja konta studenta (imię, nazwisko, adres e-mail, numer indeksu, wydział, hasło) wraz z walidacją danych,
- logowanie i wylogowanie użytkownika,
- rezerwacja sali na wybrany dzień i przedział czasowy wraz z podaniem celu rezerwacji,
- podgląd dostępności wybranej sali w danym dniu (liczba zajętych i wolnych miejsc w każdym przedziale),
- przegląd własnych rezerwacji z podziałem na nadchodzące i minione, anulowanie nadchodzących rezerwacji,
- eksport własnych rezerwacji do pliku CSV,
- edycja profilu (imię, nazwisko, e-mail, wydział) oraz zmiana hasła,
- przegląd katalogu sal z filtrowaniem po typie i minimalnej pojemności oraz z podglądem zajętości na wybrany dzień.

2.2 Wymagania niefunkcjonalne

- **Użyteczność** – prosty, responsywny interfejs w ciemnej stylistyce, czytelne komunikaty o błędach i potwierdzenia operacji,
- **Spójność danych** – ograniczenia integralności w bazie danych (klucze obce, unikalność, wyzwalacze) niezależne od warstwy aplikacji,
- **Bezpieczeństwo** – hasła nie są przechowywane jawnie (skrót SHA-256), zapytania SQL są parametryzowane (ochrona przed SQL injection), konfiguracja połączenia jest wyprowadzona do zmiennych środowiskowych,
- **Przenośność** – uruchomienie środowiska w kontenerach Docker za pomocą jednego polecenia,

- **Wydajność i skalowalność** – indeksy na najczęściej używanych kolumnach oraz możliwość wymiany warstwy dostępu do danych (bezpośrednio do bazy lub przez REST API) bez zmian w interfejsie,
- **Rozszerzalność** – modułowa budowa umożliwiająca dodawanie nowych sal, przedziałów czasowych i funkcji.

3 Przegląd istniejących rozwiązań

W niniejszym rozdziale przedstawiono analizę trzech podobnych rozwiązań, z których korzystają organizacje do zarządzania rezerwacjami sal i zasobów. Analiza ma charakter ogólny i służy wskazaniu, dlaczego zdecydowano się na własne, dopasowane do potrzeb uczelni rozwiązanie [7, 8, 9]. Zestawienie porównawcze zawiera tabela 3.

3.1 Rozwiązanie A: Kalendarz Google z zasobami

Kalendarz Google w ramach Google Workspace pozwala definiować sale i inne zasoby, które można rezerwować przez dodanie ich do wydarzenia w kalendarzu [7]. Zaletą jest głęboka integracja z pocztą, zaproszeniami i kalendarzami użytkowników oraz brak konieczności instalacji własnej infrastruktury. Wadami są zależność od platformy i licencji jednego dostawcy, ograniczone możliwości dostosowania reguł rezerwacji do specyfiki uczelni (np. stałe przedziały zajęć, model miejsc w sali) oraz brak pełnej kontroli nad strukturą i przechowywaniem danych.

3.2 Rozwiązanie B: Skrzynki zasobów Exchange / Outlook

W środowisku Microsoft 365 sale mogą być reprezentowane przez tzw. skrzynki zasobów (*room mailboxes*), które automatycznie przyjmują lub odrzucają zaproszenia na spotkania [8]. Rozwiązanie sprawdza się w organizacjach korzystających już z ekosystemu Microsoft i pozwala na automatyczne zatwierdzanie rezerwacji. Jego ograniczeniem jest konieczność posiadania odpowiedniej infrastruktury i licencji, stosunkowo złożona konfiguracja po stronie administratora oraz brak dedykowanego, prostego interfejsu skierowanego do studentów.

3.3 Rozwiązanie C: Wyspecjalizowane platformy rezerwacji (np. Skedda)

Komercyjne platformy do rezerwacji przestrzeni, takie jak Skedda, oferują rozbudowane funkcje: plany pięter, reguły dostępu, raporty wykorzystania czy powiadomienia [9]. Są to rozwiązania kompletne, ale płatne, wymagające przekazania danych zewnętrznemu dostawcy, a zakres ich funkcji znacznie wykracza poza potrzeby prostego systemu rezerwacji sal dydaktycznych.

Wnioski z przeglądu. Istniejące rozwiązania są dojrzałe, lecz albo uzależniają organizację od zewnętrznego dostawcy, albo oferują nadmiarową funkcjonalność kosztem prostoty. Własna aplikacja UniRoom pozwala dopasować model danych (stałe przedziały czasowe, katalog sal z

Tabela 3: Porównanie przeanalizowanych rozwiązań z projektem UniRoom

Cecha	A	B	C	UniRoom
Własność danych	dostawca	dostawca / organizacja	dostawca	pełna (własna baza)
Koszt	licencja	licencja	subskrypcja	brak licencji
Dopasowanie do slotów uczelni	częściowe	częściowe	konfigurowalne	pełne
Prostota interfejsu dla studenta	średnia	średnia	wysoka	wysoka
Wymagana infrastruktura	konto Workspace	Microsoft 365 / Exchange	konto w usłudze	Docker
Możliwość rozbudowy kodu	brak	ograniczona	brak	pełna

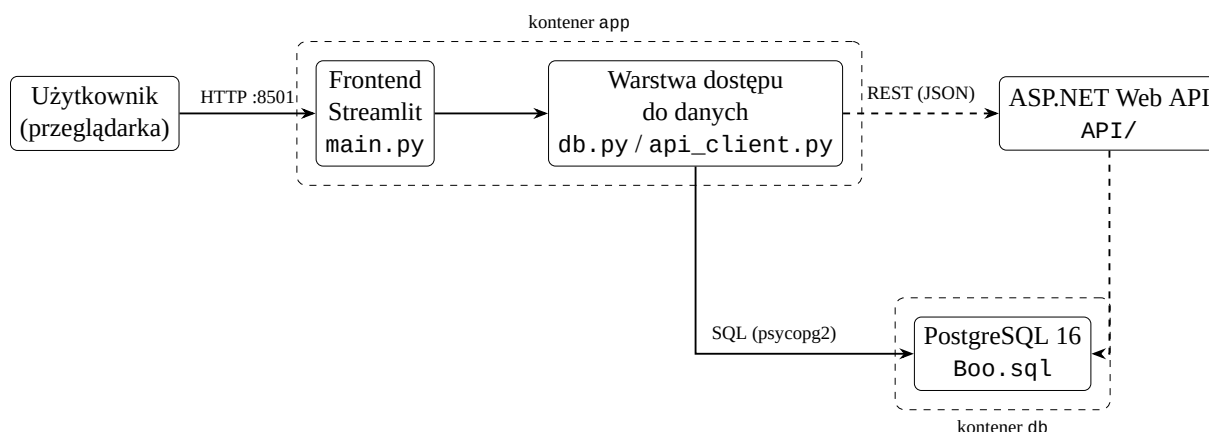
wyposażeniem, model miejsc w sali) do potrzeb uczelni oraz w pełni kontrolować dane, a także stanowi dobry poligon do zastosowania technologii poznawanych w ramach przedmiotu.

4 Opis architektury

System ma architekturę warstwową i składa się z trzech głównych warstw: prezentacji, logiki i dostępu do danych oraz warstwy danych. Całość jest uruchamiana w kontenerach Docker. Tabela 4 przedstawia zestawienie głównych komponentów wraz z ich rolą w systemie, a rysunek 1 – ich wzajemne powiązania.

Tabela 4: Główne komponenty architektury systemu

Lp.	Komponent	Warstwa	Technologia i rola
1	Frontend Web	Prezentacja	Streamlit (<code>main.py</code>); interfejs użytkownika, walidacja formularzy, stan sesji
2	Backend (dostęp do danych)	Logika	<code>db.py</code> (psycopg2, bezpośrednio do bazy) lub <code>api_client.py</code> (requests, przez REST API)
3	Backend API	Logika	ASP.NET Web API (katalog API/); punkty końcowe <code>/api/users</code> i <code>/api/bookings</code>
4	Baza danych	Dane	PostgreSQL 16 (<code>Boo.sql</code>); tabele, widoki, funkcje, wyzwalacze
5	Środowisko uruchomieniowe	Infrastruktura	Docker i Docker Compose; kontenery app oraz db



Rysunek 1: Architektura systemu UniRoom (linia przerywana – alternatywna ścieżka przez REST API)

4.1 Podział na warstwy

Warstwa prezentacji. Aplikacja Streamlit renderuje pięć stron: rezerwacja sali, moje rezerwacje, mój profil, przegląd sal oraz informacje o systemie. Stan zalogowanego użytkownika,

lista użytkowników i lista rezerwacji są przechowywane w `st.session_state`, a po każdej operacji zapisującej dane są odświeżane z bazy.

Warstwa logiki i dostępu do danych. Interfejs komunikuje się z danymi wyłącznie przez moduł o ujednoliconym API (`get_all_users`, `create_user`, `update_user`, `update_password`, `get_all_bookings`, `add_booking`, `cancel_booking`). Istnieją dwie wymienne implementacje tego interfejsu: `db.py` łączy się bezpośrednio z PostgreSQL, natomiast `api_client.py` wywołuje REST API napisane w ASP.NET. Przełączenie polega na zmianie jednej linii importu w `main.py` (`import api_client as db`), co realizuje wzorzec adaptera i utrzymuje luźne powiązanie warstw.

Warstwa danych. PostgreSQL przechowuje słowniki (wydziały, typy sal, przedziały czasowe), sale wraz z wyposażeniem, użytkowników oraz rezerwacje. Reguły integralności są egzekwowane po stronie bazy.

4.2 Przepływ danych – przykład rezerwacji sali

1. Użytkownik wybiera salę, datę i przedział czasowy na stronie *Book a Room*; lista dostępnych przedziałów jest wyliczana na podstawie rezerwacji załadowanych do stanu sesji.
2. Po kliknięciu *Confirm Booking* aplikacja sprawdza, czy użytkownik nie ma już rezerwacji w tym terminie oraz czy przedział nie jest pełny.
3. Aplikacja buduje słownik rezerwacji (identyfikator, dane użytkownika, cel, sala, data, przedział, czas utworzenia) i wywołuje `add_booking`.
4. Warstwa danych wykonuje parametryzowany `INSERT` do tabeli `bookings`, przy czym identyfikator przedziału jest ustalany na podstawie jego etykiety.
5. Baza danych weryfikuje ograniczenia (klucze obce, unikalność, wyzwalacze); w razie naruszenia operacja jest wycofywana, a użytkownik otrzymuje komunikat o błędzie.
6. Po powodzeniu lista rezerwacji jest odświeżana z bazy, a widok dostępności aktualizowany.

5 Opis implementacji

W rozdziale omówiono kluczowe aspekty implementacyjne projektu. Opis został podzielony na sześć części obejmujących frontend, backend, bazę danych, konteneryzację, testy oraz monitoring. Przy każdej części wskazano osobę odpowiedzialną za jej wykonanie.

5.1 Frontend

Odpowiedzialny: Dmytro Kovalov.

Interfejs użytkownika został wykonany w języku Python z użyciem biblioteki Streamlit [2], która pozwala budować aplikacje webowe bez ręcznego pisania kodu HTML i JavaScript. Całość logiki interfejsu znajduje się w pliku `main.py`.

5.1.1 Konfiguracja i stylizacja

Aplikacja ustawia tytuł strony, ikonę i szeroki układ (`layout="wide"`) za pomocą `st.set_page_config`. Wygląd został dopracowany za pomocą własnego arkusza CSS wstrzykiwanego przez `st.markdown(..., unsafe_allow_html=True)`: ciemna kolorystyka, czcionki Playfair Display i DM Sans, gradientowe przyciski, karty sal, kafelki rezerwacji oraz znaczniki dostępności (*Free*, *Full*).

5.1.2 Stan sesji i uwierzytelnianie

Do przechowywania stanu wykorzystano `st.session_state`: listę użytkowników, listę rezerwacji, flagę zalogowania oraz identyfikator bieżącego użytkownika. Dopóki użytkownik nie jest zalogowany, aplikacja wyświetla wyłącznie ekran uwierzytelniania (dwie zakładki: logowanie i rejestracja) i przerywa dalsze renderowanie przez `st.stop()`.

Walidacje po stronie interfejsu obejmują:

- wypełnienie wszystkich pól formularza,
- poprawność adresu e-mail (wyrażenie regularne),
- unikalność numeru indeksu,
- minimalną długość hasła (6 znaków) i zgodność powtórzonego hasła,
- hashowanie hasła algorytmem SHA-256 (`hashlib`) przed zapisem lub porównaniem.

5.1.3 Strony aplikacji

Book a Room

Formularz rezerwacji: cel, sala, data (od dziś do 30 dni w przód) oraz przedział czasowy z informacją o liczbie wolnych miejsc. Obok formularza wyświetlana jest karta sali (piętro, pojemność, typ, wyposażenie) i pasek zajętości każdego przedziału w wybranym dniu z kolorowym wskaźnikiem (zielony, pomarańczowy, czerwony).

My Bookings

Statystyki (razem, nadchodzące, minione), zakładki *Upcoming* i *Past* z kafelkami rezerwacji, przycisk anulowania dla rezerwacji nadchodzących oraz eksport do CSV z użyciem biblioteki pandas.

My Profile

Edycja imienia, nazwiska, e-maila i wydziału, zmiana hasła (z weryfikacją starego hasła) oraz metryki aktywności użytkownika.

Room Overview

Katalog sal z filtrami (typ, minimalna pojemność, data) i podglądem zajętości poszczególnych przedziałów.

About

Zasady działania systemu, reguły rezerwacji i tabela przedziałów czasowych.

5.1.4 Model sal i przedziałów czasowych

Katalog sal (budynki P i K, po dziewięć sal na trzech piętrach) oraz lista siedmiu przedziałów czasowych (od 08:00–09:30 do 18:30–20:15) są zdefiniowane w postaci słowników i list na początku `main.py`. Każda sala ma typ (*hall*, *lab*, *seminar*, *study*), piętro, pojemność i listę wyposażenia. Przedział czasowy może pomieścić do 30 rezerwacji (stała `MAX_SEATS`), co pozwala odwzorować rezerwację miejsca w sali, a nie tylko całej sali.

5.2 Backend

Odpowiedzialny: *Dmytro Kovalov*.

Warstwa backendowa składa się z trzech elementów: modułu `db.py`, klienta REST `api_client.py` oraz usługi ASP.NET Web API w katalogu `API/`.

5.2.1 Moduł `db.py`

Moduł `db.py` realizuje bezpośredni dostęp do PostgreSQL za pomocą biblioteki `psycopg2` [4]. Parametry połączenia (host, port, nazwa bazy, użytkownik, hasło) są wczytywane ze zmiennych

środowiskowych przy użyciu `python-dotenv`, z wartościami domyślnymi dla środowiska lokalnego. Moduł udostępnia funkcje:

- `get_all_users()` – zwraca słownik {numer indeksu: dane użytkownika}, łącząc tabele `users` i `faculties`,
- `create_user(...)` – dodaje użytkownika; wydział jest rozwiązywany podzapytaniem po nazwie; przy naruszeniu unikalności (indeks lub e-mail) transakcja jest wycofywana, a funkcja zwraca `False`,
- `update_user(...)` i `update_password(...)` – aktualizacja profilu i hasła,
- `get_all_bookings()` – zwraca aktywne rezerwacje (bez anulowanych) wraz z danymi użytkownika, wydziału i przedziału, posortowane po dacie i godzinie,
- `add_booking(...)` – wstawia rezerwację, ustalając identyfikator przedziału na podstawie jego etykiety (z normalizacją znaku myślnika); zwraca `False`, jeśli przedziału nie znaleziono lub naruszono ograniczenia,
- `cancel_booking(...)` – tzw. miękkie usunięcie: ustawia znacznik czasu `cancelled_at` zamiast fizycznego usuwania rekordu.

Wszystkie zapytania są parametryzowane, co chroni przed atakami typu SQL injection.

5.2.2 Klient REST `api_client.py`

Moduł `api_client.py` udostępnia ten sam zestaw funkcji co `db.py`, ale zamiast łączyć się z bazą, wysyła żądania HTTP (biblioteka `requests`, limit czasu 10 s) do usługi ASP.NET. Adres usługi pochodzi ze zmiennej `API_BASE_URL`. Zwracane struktury mają identyczny format jak w `db.py`, dzięki czemu `main.py` nie wymaga zmian przy przełączaniu implementacji. Wykorzystywane punkty końcowe zestawiono w tabeli 5.

Tabela 5: Punkty końcowe REST API wykorzystywane przez `api_client.py`

Metoda	Ścieżka	Opis
GET	<code>/api/users</code>	lista użytkowników
POST	<code>/api/users</code>	rejestracja użytkownika (kod 201 przy sukcesie)
PUT	<code>/api/users/{student_id}</code>	aktualizacja danych profilu
PATCH	<code>/api/users/{student_id}/password</code>	zmiana hasła (skrót)
GET	<code>/api/bookings</code>	lista aktywnych rezerwacji
POST	<code>/api/bookings</code>	utworzenie rezerwacji (kod 201 przy sukcesie)
DELETE	<code>/api/bookings/{booking_id}</code>	anulowanie rezerwacji

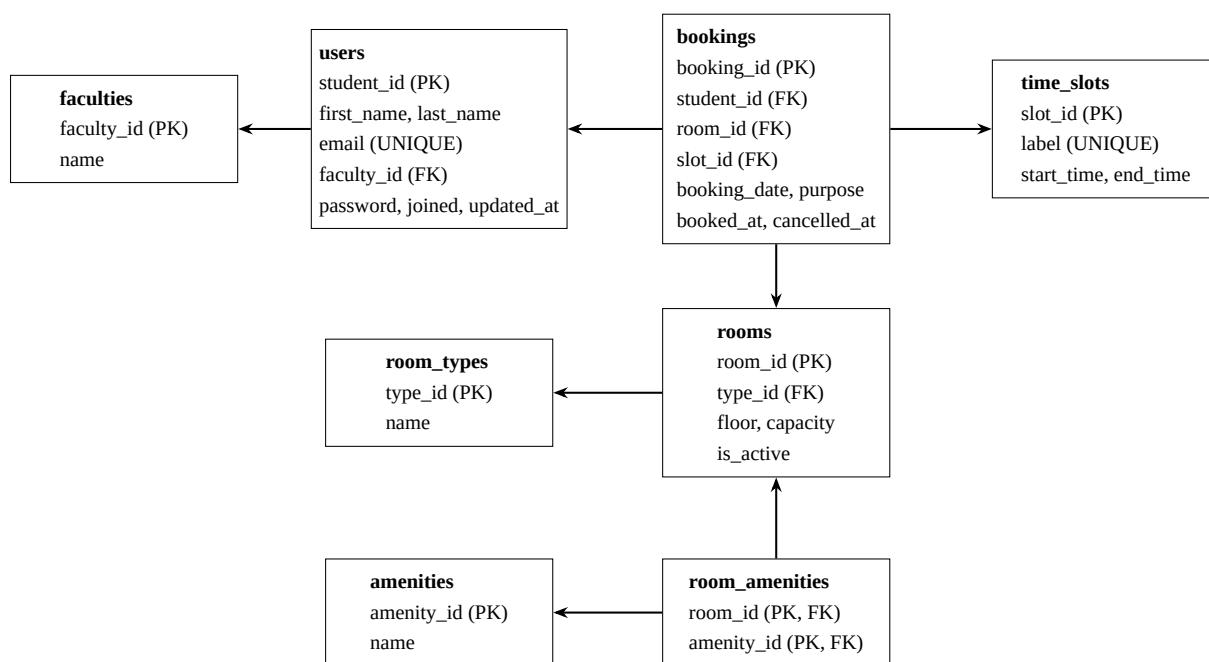
5.2.3 Usługa ASP.NET Web API

Katalog API/ zawiera usługę *BookingApi* napisaną w języku C# z użyciem ASP.NET Core [6]. Udostępnia ona opisane wyżej punkty końcowe i stanowi warstwę pośrednią pomiędzy klientem a bazą danych; obiekty transferowe (*UserDto*, *BookingDto*) odpowiadają strukturze danych używanej w interfejsie. Katalog obj/ w repozytorium zawiera artefakty kompilacji .NET.

5.3 Baza danych

Odpowiedzialny: Pavlo Krasnivskyi.

Warstwę danych zaprojektowano w PostgreSQL 16 [3]. Pełny schemat wraz z danymi początkowymi znajduje się w pliku *Boo.sql* i odpowiada strukturom używanym w aplikacji. Rysunek 2 przedstawia uproszczony diagram związków encji.



Rysunek 2: Uproszczony diagram związków encji bazy danych booking_san

5.3.1 Struktura tabel

Tabela 6: Tabele bazy danych

Tabela	Opis
faculties	słownik wydziałów (10 pozycji zgodnych z listą w aplikacji)
room_types	słownik typów sal: <i>hall</i> , <i>lab</i> , <i>seminar</i> , <i>study</i>
time_slots	przedziały czasowe z etykietą oraz godziną rozpoczęcia i zakończenia (ograniczenie: koniec po początku)
rooms	18 sal w budynkach P i K: typ, piętro, pojemność, flaga aktywności
amenities, room_amenities	wyposażenie sal i relacja wiele-do-wielu
users	konta studentów; klucz główny to numer indeksu, unikalny e-mail, hasło jako skrót SHA-256 (64 znaki hex)
bookings	rezerwacje; miękkie anulowanie przez <code>cancelled_at</code>

5.3.2 Integralność danych

Integralność zapewniono w kilku warstwach:

- **klucze obce** pomiędzy `users`, `bookings`, `rooms`, `time_slots` i słownikami; usunięcie użytkownika kasuje kaskadowo jego rezerwacje,
- **ograniczenie unikalności** (`room_id`, `booking_date`, `slot_id`) w tabeli `bookings`,
- **ograniczenia CHECK** (np. piętro w zakresie 1–10, dodatnia pojemność, poprawna kolejność godzin),
- **wyzwalacz** blokujący rezerwację na datę z przeszłości,
- **wyzwalacz** blokujący rezerwację sali oznaczonej jako nieaktywna,
- **wyzwalacz** automatycznie aktualizujący pole `updated_at` w tabeli `users`,
- **indeksy** na kolumnach `student_id`, (`room_id`, `booking_date`) oraz `booking_date`.

5.3.3 Widoki i funkcje

Aby uprościć zapytania z warstwy aplikacji, przygotowano widoki: `users_view` (użytkownik z pełnym imieniem i nazwą wydziału), `bookings_full` (pełne dane rezerwacji wraz z typem

sali, pięciem i statusem), `plan_dnia` (rezerwacje na bieżący dzień) oraz `statystyki_sal` (liczba rezerwacji na salę). Zdefiniowano także funkcje pomocnicze odpowiadające funkcjom z kodu aplikacji: `czy_slot_zajety`, `rezerwacje_sali`, `wolne_sloty` oraz `anuluj_rezerwacje`, która dodatkowo zabrania anulowania rezerwacji z przeszłości.

Plik `Boo.sql` zawiera również zestaw przykładowych zapytań dokumentujących typowe operacje (rejestracja, logowanie, rezerwacja, anulowanie, statystyki profilu) oraz szkic skryptu migracji danych z plików JSON, z których korzystała wcześniejsza wersja aplikacji.

5.4 Docker

Odpowiedzialny: Pavlo Krasnivskyi.

Konteneryzacja pozwala uruchomić całe środowisko na dowolnym komputerze z zainstalowanym Dockerem [5], bez ręcznej instalacji Pythona i PostgreSQL.

5.4.1 Obraz aplikacji (Dockerfile)

Obraz aplikacji jest budowany na bazie `python:3.11-slim`. Kolejne kroki to: ustawienie katalogu roboczego `/app`, skopiowanie i instalacja zależności z `requirements.txt` (przed skopiowaniem kodu, co pozwala wykorzystać pamięć podręczną warstw Dockera), skopiowanie kodu źródłowego, udostępnienie portu 8501 i uruchomienie aplikacji poleceniem `streamlit run main.py --server.address=0.0.0.0`.

5.4.2 Orkiestracja (docker-compose.yml)

Plik `docker-compose.yml` definiuje dwie usługi:

- `db` – obraz `postgres:16`; nazwa bazy i użytkownik ustawiane są zmiennymi środowiskowymi; dane są trwale przechowywane w wolumenie `postgres_data`; plik `Boo.sql` jest montowany do katalogu `/docker-entrypoint-initdb.d/`, dzięki czemu schemat i dane słownikowe są tworzone automatycznie przy pierwszym uruchomieniu; port 5432 jest udostępniony na hosta,
- `app` – obraz budowany z lokalnego `Dockerfile`; port 8501 udostępniony na hosta; konfiguracja pobierana z pliku `.env`; usługa startuje po usłudze `db` (`depends_on`).

5.4.3 Uruchomienie

```
git clone https://github.com/Dimkov57/Booking_SAN.git
cd Booking_SAN
docker compose up --build
# aplikacja: http://localhost:8501
```

W środowisku kontenerowym zmienna `DB_HOST` w pliku `.env` powinna wskazywać nazwę usługi bazy (db), a nie `localhost`.

5.5 Testy

Odpowiedzialny: Dmytro Kovalov i Pavlo Krasnivskyi (wspólnie).

Testowanie było wspólnym obszarem odpowiedzialności obu autorów. Przyjęto podział zgodny z podziałem prac: Dmytro Kovalov weryfikował interfejs i warstwę backendową, Pavlo Krasnivskyi – bazę danych oraz środowisko Docker; działanie całości sprawdzono wspólnie.

5.5.1 Zakres i poziomy testów

- **testy funkcjonalne interfejsu** – przejście przez wszystkie strony aplikacji i scenariusze użytkownika, w tym ścieżki błędów (niepoprawne dane, próba podwójnej rezerwacji),
- **testy warstwy backendowej** – poprawność funkcji dostępu do danych oraz obsługi błędów (naruszenie unikalności, brak przedziału czasowego),
- **testy bazy danych** – weryfikacja ograniczeń, wyzwalaczy i funkcji na danych testowych, m.in. za pomocą przykładowych zapytań z `Boo.sql`,
- **testy środowiska** – czysty start środowiska poleceniem `docker compose up --build`, inicjalizacja bazy, trwałość danych po restarcie kontenerów,
- **testy integracyjne** – kompletny przepływ: rejestracja, logowanie, rezerwacja, anulowanie, wylogowanie.

5.5.2 Scenariusze testowe

Tabela 7 zawiera przyjęte scenariusze testowe wraz z oczekiwanymi rezultatami.

Tabela 7: Scenariusze testowe

ID	Obszar	Scenariusz	Oczekiwany rezultat
T01	Frontend	Rejestracja z poprawnymi danymi	konto utworzone, użytkownik zalogowany
T02	Frontend	Rejestracja z istniejącym numerem indeksu	komunikat o zajętych numerze, brak nowego konta
T03	Frontend	Rejestracja z niepoprawnym adresem e-mail	komunikat o niepoprawnym adresie

ID	Obszar	Scenariusz	Oczekiwany rezultat
T04	Frontend	Hasło krótsze niż 6 znaków lub różne hasła	komunikat o błędzie, brak rejestracji
T05	Frontend	Logowanie z poprawnymi danymi	przejsięcie do aplikacji
T06	Frontend	Logowanie z błędnym hasłem lub nieznanym indeksem	odpowiedni komunikat, brak logowania
T07	Frontend	Rezerwacja wolnego przedziału	rezerwacja zapisana i widoczna w <i>My Bookings</i>
T08	Frontend	Ponowna rezerwacja tego samego terminu przez tego samego użytkownika	komunikat o istniejącej rezerwacji
T09	Backend	Rezerwacja przedziału o wyczerpanym limicie miejsc	przedział niedostępny do wyboru lub komunikat o braku miejsc
T10	Backend	Anulowanie nadchodzącej rezerwacji	rezerwacja znika z listy aktywnych, w bazie ustawione <code>cancelled_at</code>
T11	Frontend	Edycja profilu (poprawne i niepoprawne dane)	zapis zmian lub komunikat walidacyjny
T12	Frontend	Zmiana hasła (błędne stare hasło, zbyt krótkie nowe)	odrzućenie zmiany z komunikatem; poprawne dane – zmiana zapisana
T13	Frontend	Filtrowanie w <i>Room Overview</i> (typ, pojemność, data)	lista sal zgodna z filtrami
T14	Frontend	Eksport rezerwacji do CSV	pobrany plik zawiera rezerwacje użytkownika
T15	Frontend	Wybór daty z przeszłości lub dalej niż 30 dni	wybór niemożliwy (ograniczenie kalendarza)
T16	Baza danych	Wstawienie rezerwacji na przeszłą datę bezpośrednio w SQL	wyjątek zgłoszony przez wyzwalacz
T17	Baza danych	Wstawienie rezerwacji na nieaktywną salę	wyjątek zgłoszony przez wyzwalacz
T18	Baza danych	Duplikat rezerwacji (sala, data, przedział)	naruszenie ograniczenia unikalności

ID	Obszar	Scenariusz	Oczekiwany rezultat
T19	Baza danych	Usunięcie użytkownika z rezerwacjami	rezerwacje usunięte kaskadowo
T20	Docker	Uruchomienie od zera (docker compose up --build)	obie usługi startują, schemat bazy utworzony, aplikacja dostępna na porcie 8501
T21	Docker	Restart kontenerów	dane zachowane dzięki wolumenowi

5.6 Monitoring

Odpowiedzialny: Pavlo Krasnivskyi (w zakresie środowiska Docker).

W projekcie zastosowano podstawowy monitoring oparty na mechanizmach dostarczanych przez środowisko kontenerowe i użyte komponenty:

- **logi kontenerów** – podgląd na bieżąco poleceniem `docker compose logs -f app` (aplikacja Streamlit) oraz `docker compose logs -f db` (PostgreSQL), w tym błędy połączeń i naruszenia ograniczeń,
- **stan usług** – polecenie `docker compose ps` pokazuje status kontenerów, a `docker stats` – zużycie CPU i pamięci,
- **historia rezerwacji w bazie** – dzięki miękkiemu anulowaniu i znacznikom `booked_at` oraz `cancelled_at` możliwa jest analiza aktywności użytkowników; widok `statystyki_sal` agreguje wykorzystanie sal,
- **obsługa błędów po stronie aplikacji** – komunikaty dla użytkownika oraz kontrolowane wycofanie transakcji przy błędach zapisu.

Rozszerzeniem, możliwym w kolejnych wersjach, jest dodanie kontrolek stanu (healthcheck) w `docker-compose.yml` oraz zebranie metryk w systemie Prometheus z wizualizacją w Grafanie.

6 Podsumowanie i wnioski

6.1 Rezultaty

Zrealizowano działającą aplikację UniRoom, która umożliwia studentom zakładanie kont, przeglądanie sal, sprawdzanie dostępności oraz rezerwowanie i anulowanie terminów. Powstał kompletny stos technologiczny: interfejs w Streamlit, warstwa dostępu do danych z dwiema wymiennymi implementacjami (bezpośrednio do PostgreSQL oraz przez REST API w ASP.NET), relacyjna baza danych z ograniczeniami, widokami, funkcjami i wyzwalaczami oraz konteneryzacja w Docker Compose. Podział prac (Dmytro Kovalov – frontend, backend, testy; Pavlo Krasnivskyi – baza danych, Docker, testy) pozwolił na równoległą pracę nad warstwami dzięki wcześniej uzgodnionemu formatowi danych wymienianych między nimi.

6.2 Napotkane problemy i sposoby ich rozwiązania

- **Spójność danych między aplikacją a bazą.** Aplikacja i baza były rozwijane równolegle, dlatego wymagały uzgodnienia formatu danych (np. etykiety przedziałów czasowych z różnymi znakami myślnika – rozwiązano przez normalizację w zapytaniu). Wskazano także obszary do ujednolicenia: liczba przedziałów czasowych w aplikacji i w słowniku bazy, nazwy wyposażenia sal (język angielski w aplikacji, polski w bazie) oraz model miejsc w sali (limit 30 rezerwacji w aplikacji względem ograniczenia unikalności terminu w bazie).
- **Wymienność warstwy dostępu do danych.** Wprowadzono jednolity interfejs funkcji w `db.py` i `api_client.py`, dzięki czemu zmiana sposobu komunikacji z danymi nie wpływa na interfejs użytkownika.
- **Konfiguracja środowiska.** Różnice pomiędzy uruchomieniem lokalnym a kontenerowym (adres hosta bazy) rozwiązano przez zmienne środowiskowe i plik `.env`.
- **Inicjalizacja bazy.** Skrypt `Boo.sql` uruchamiany automatycznie przy pierwszym starcie kontenera powinien zawierać wyłącznie poprawny kod SQL; fragmenty pomocnicze (np. skrypt migracji w Pythonie) należy przechowywać w osobnych plikach.

6.3 Kierunki dalszego rozwoju

- bezpieczniejsze przechowywanie haseł (algorytmy z solą, np. `bcrypt` lub `Argon2`) oraz przeniesienie danych dostępowych z plików konfiguracyjnych repozytorium do mechanizmu sekretów,

- pobieranie katalogu sal i wyposażenia z bazy danych zamiast definiowania go w kodzie aplikacji,
- dodanie usługi REST API do pliku `docker-compose.yml` jako trzeciego kontenera,
- automatyzacja testów (pytest, testy integracyjne z bazą w kontenerze) i potok CI w GitHub Actions,
- rola administratora (zarządzanie salami, blokowanie terminów), powiadomienia e-mail o rezerwacjach,
- monitoring z użyciem `healthcheck`, Prometheusa i Grafany.

Spis tabel

1	Podział odpowiedzialności w zespole projektowym	3
2	Przyporządkowanie elementów repozytorium do autorów	4
3	Porównanie przeanalizowanych rozwiązań z projektem UniRoom	8
4	Główne komponenty architektury systemu	9
5	Punkty końcowe REST API wykorzystywane przez <code>api_client.py</code>	13
6	Tabele bazy danych	15
7	Scenariusze testowe	17

Spis rysunków

1	Architektura systemu UniRoom (linia przerywana – alternatywna ścieżka przez REST API)	9
2	Uproszczony diagram związków encji bazy danych booking_san	14

Bibliografia

- [1] Dmytro Kovalov, Pavlo Krasnivskyi. *Booking_SAN* – repozytorium projektu. url: https://github.com/Dimkov57/Booking_SAN (term. wiz. 23.09.2026).
- [2] Streamlit Inc. *Streamlit Documentation*. url: <https://docs.streamlit.io> (term. wiz. 23.09.2026).
- [3] PostgreSQL Global Development Group. *PostgreSQL 16 Documentation*. url: <https://www.postgresql.org/docs/16/> (term. wiz. 23.09.2026).
- [4] Psycopg. *Psycopg 2 Documentation*. url: <https://www.psycopg.org/docs/> (term. wiz. 23.09.2026).
- [5] Docker Inc. *Docker Documentation*. url: <https://docs.docker.com> (term. wiz. 23.09.2026).
- [6] Microsoft. *ASP.NET Core documentation*. url: <https://learn.microsoft.com/aspnet/core> (term. wiz. 23.09.2026).
- [7] Google. *Google Calendar (Google Workspace)*. url: <https://workspace.google.com/products/calendar/> (term. wiz. 23.09.2026).
- [8] Microsoft. *Room mailboxes in Exchange*. url: <https://learn.microsoft.com/exchange/recipients/room-mailboxes> (term. wiz. 23.09.2026).
- [9] Skedda. *Skedda – space booking software*. url: <https://www.skedda.com> (term. wiz. 23.09.2026).